

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

CS3807 Deep Learning Laboratory

B.Tech Artificial Intelligence and Data Science, Semester V

1 Objective

The objective of this experiment is to build an end to end understanding of recurrent sequence learning by implementing and comparing a Vanilla RNN, an LSTM and a GRU on the same task. The experiment also does Backpropagation through the exp and the vanishing and exploding gradient problem, extends the recurrent pipeline to video understanding using CNN extracted features and it also studies the encoder decoder framework by a small sequence to sequence task. The UCI Human Activity Recognition Using Smartphones dataset is the primary sequence dataset.

2 Dataset and Experimental Setup

The UCI HAR dataset contains smartphone sensor classifications for six activities: WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING and LAYING. The raw inertial signal files were used rather than the precomputed 561 feature vectors. Each window has 128 time steps and nine channels, made up of three axis body acceleration, three axis gyroscope and three axis total acceleration. The input representation is hence

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

We take a balanced laboratory subset with 2400 windows was used, with 400 windows per class drawn from the full training partition. The full training and test partitions loaded as (7352, 128, 9) and (2947, 128, 9) respectively.

Table 1: Input tensor dimensions.

Dimension	Meaning	Value
N	Number of sequences	2400 (subset)
T	Time steps per sequence	128
F	Features per time step	9

3 Preprocessing

The subset was split into 70 percent training, 15 percent validation , 15 percent testing using stratified sampling so all six classes stay equal(The total in each one). The input was normalized per channel using statistics computed only from the training data.

Table 2: Preprocessing output shapes.

Set	Shape
Training	(1680,128,9)
Validation	(360,128,9)
Testing	(360,128,9)
Number of classes	6
Features per time step	9
Sequence length	128

The training class distribution is 280 windows per class.

4 Temporal Data Visualization

Representative sequences from three activity classes were plotted for three sensor channels over the 128 time steps. Figure 1 shows the result.

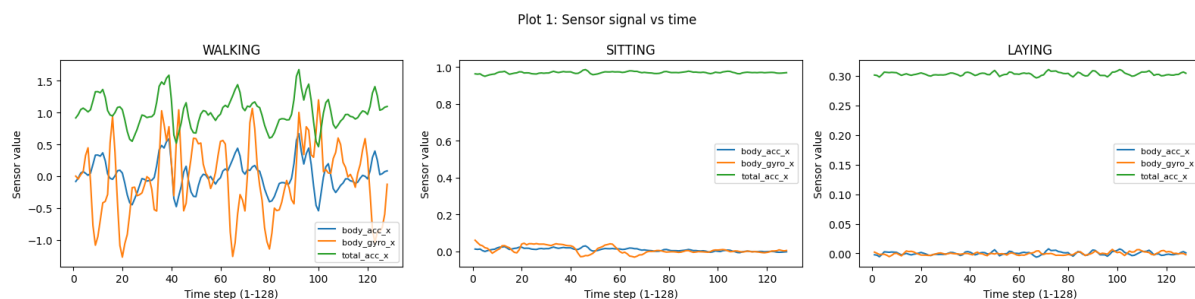


Figure 1: Plot 1: Sensor signal versus time for WALKING, SITTING and LAYING.

Inference. The walking activity shows repeated oscillations in the acceleration and gyroscope channels, because walking is a periodic motion(Most of the time). The sitting and laying activities in the graph are more straight since the body won't be moving. Activities that involve movement look similar among each other and static activities look similar among each other. The ordering of the measurements matters because the class is defined by how the values change over time. If the 128 steps were shuffled the periodic pattern would be lost and the activity could no longer be told apart.

5 Backpropagation Through Time

When an RNN is unrolled. The loss depends on the entire chain of hidden states and BPTT propagates the error backward through every time step with repeated multiplication derivative terms across all the steps can make the gradient shrink toward zero, which is the vanishing gradient problem, or grow very large, which is the exploding gradient problem. This is why a plain RNN struggles to learn dependencies in long sequences.

A numerical check was done using $x_1 = 0.5$, $x_2 = 0.7$, $x_3 = 0.2$, $h_0 = 0$, $W_x = 0.5$, $W_h = 0.8$, $b = 0.1$ and $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$. The hand calculated values matched the program values exactly.

Table 3: Manual versus program hidden state values.

Step	Manual	Program	Difference
h_1	0.336376	0.336376	0
h_2	0.616352	0.616352	0
h_3	0.599958	0.599958	0

Inference. The manual and program values agree, which confirms the forward recurrence was implemented correctly. Since each tanh derivative is below 1, multiplying many of them together across long sequences pushes the gradient toward zero, which is the reason a Vanilla RNN loses long term information.(vanishing gradient when $\downarrow 1$)

6 Model Architectures and Training

The same classifier was used for all 3 models. Only the recurrent layer was changed between SimpleRNN, LSTM and GRU. Every model used 32 recurrent units, a dropout of 0.2, a dense layer of 16 ReLU units, a softmax output of six classes, the Adam optimizer with learning rate 10^{-3} , a batch size of 32, 30 epochs and sparse categorical cross entropy loss. Keeping these settings identical makes the comparison controlled, So basically all the difference in results comes from the recurrent layer and not from the training setup.

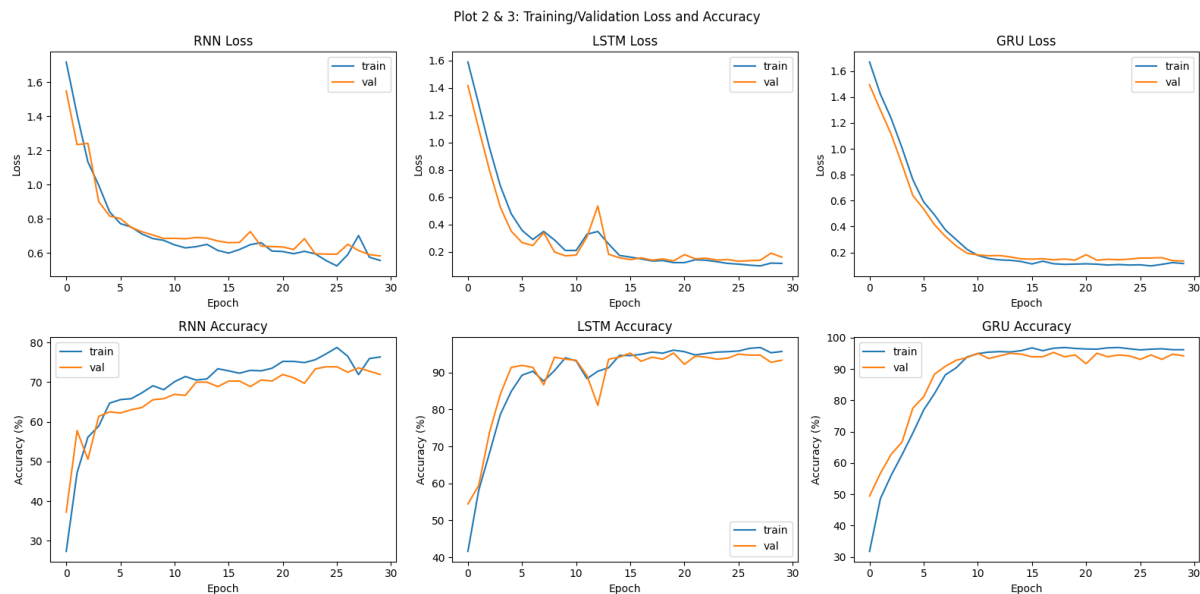


Figure 2: Plots 2 and 3: Training and validation loss and accuracy for RNN, LSTM and GRU.

Inference. The LSTM and GRU losses fall quickly and settle at low values, and their validation accuracy climbs above 90 percent, which shows good convergence with a small generalization gap. The Vanilla RNN converges more slowly and settles at a higher loss and lower accuracy. There is no strong overfitting for any model, since the validation curves track the training curves fairly closely. The RNN mainly underfits compared to the gated models.

7 Performance Evaluation

Each model was evaluated on the untouched test set. Macro averaged precision, recall and F1 were reported along with the parameter count and training time.

Table 4: Test set performance of the three recurrent models.

Metric	RNN	LSTM	GRU
Accuracy (%)	76.67	94.17	95.00
Macro Precision (%)	76.64	94.32	95.01
Macro Recall (%)	76.67	94.17	95.00
Macro F1 (%)	76.59	94.15	94.98
Parameters	1974	6006	4758
Training Time (s)	105.7	14.3	13.5

Inference. The GRU gave the highest test accuracy and F1, with the LSTM very close behind, while the Vanilla RNN was well below both. The GRU reached this with fewer parameters than the LSTM. The RNN took much longer to train in this run even though it has the fewest parameters, which comes from the way the plain recurrence executes rather than from model size.

8 Confusion Matrix Analysis

A separate confusion matrix was produced for each model. The label mapping is 0 WALKING, 1 WALKING_UPSTAIRS, 2 WALKING_DOWNSTAIRS, 3 SITTING, 4 STANDING, 5 LAYING.

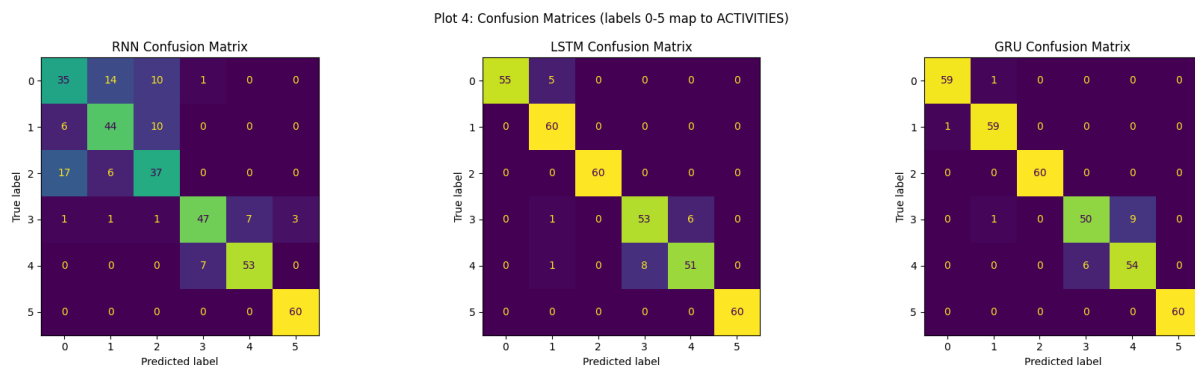


Figure 3: Plot 4: Confusion matrices for RNN, LSTM and GRU.

Inference. The activities(the ones which are static) SITTING and STANDING are the ones most often confused with each other. This may be because their sensor signatures are almost the same when the body is not moving. LAYING and the walking classes are recognized properly. The same confusion pattern shows up across all three models. This suggests the difficulty comes from the data rather than from a particular architecture. Accuracy alone would hide which classes are being mixed up, and the confusion matrix makes that visible.

9 RNN versus LSTM versus GRU Comparison

Table 5: Structural and measured comparison of the three models.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time (s)	105.7	14.3	13.5
Test F1 (%)	76.6	94.2	95.0

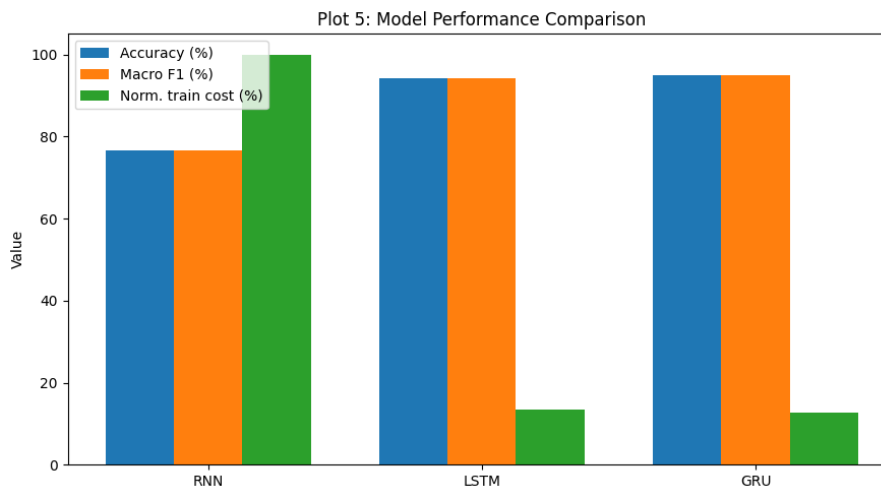


Figure 4: Plot 5: Test accuracy, macro F1 and normalized training cost.

Inference. from the graph above, it shows the trade off between predictive performance, model complexity and training cost. The Vanilla RNN is the simplest with the fewest parameters but has the weakest performance. The LSTM has the most parameters because of its extra gates and cell state (LSTM is always expensive). The GRU sits in between on complexity while matching or slightly beating the LSTM on this dataset, which makes it the best default choice here.

10 Effect of Sequence Length

The experiment was repeated with sequence lengths $T \in \{32, 64, 128\}$, keeping the last T time steps of each window.

Table 6: Test macro F1 for different sequence lengths.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	73.45	94.42	93.87
64	77.72	94.97	92.76
128	71.27	96.11	95.54

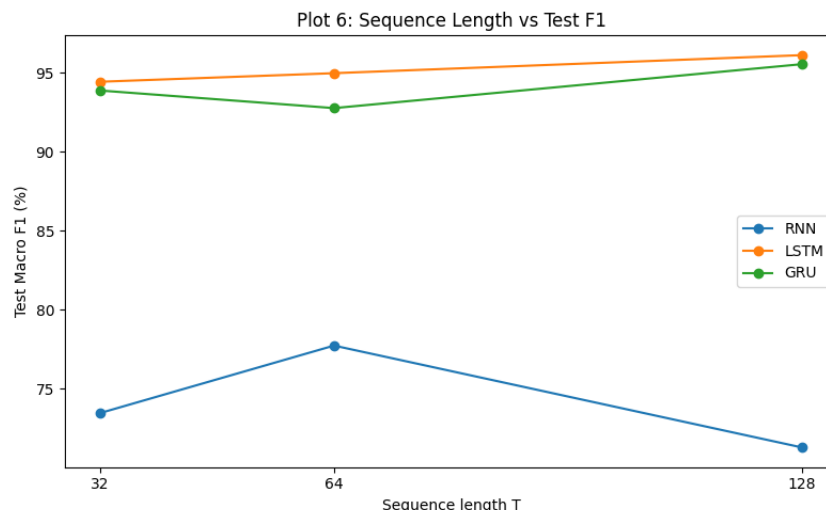


Figure 5: Plot 6: Sequence length versus test F1.

Inference. The gated models gain the most from a longer sequence, and the LSTM and GRU reach their best F1 at the full length of 128 where the window covers a complete activity cycle. The Vanilla RNN does not benefit in the same way and is actually a little worse at 128, which fits the vanishing gradient issue over long sequences. More temporal context helps as long as the model can carry information across the steps.

11 Video Understanding Using CNN and RNN

A video is a sequence of frames, so a CNN was used to extract spatial features from each frame and a recurrent model was used to learn the temporal relationship across frames. Ten frames were sampled per video at $224 \times 224 \times 3$, and a frozen MobileNetV2 with the classification head removed and global average pooling was used as the feature extractor. The CNN produced a feature dimension of 1280, giving a tensor of shape $(B, 10, 1280)$ for the recurrent network. I used 4 UCF101 action classes, Basketball, Biking, TennisSwing and WalkingWithDog, with 110 clips per class, giving us a total of 440 videos.



Figure 6: Plot 7: Sampled frames from one video.

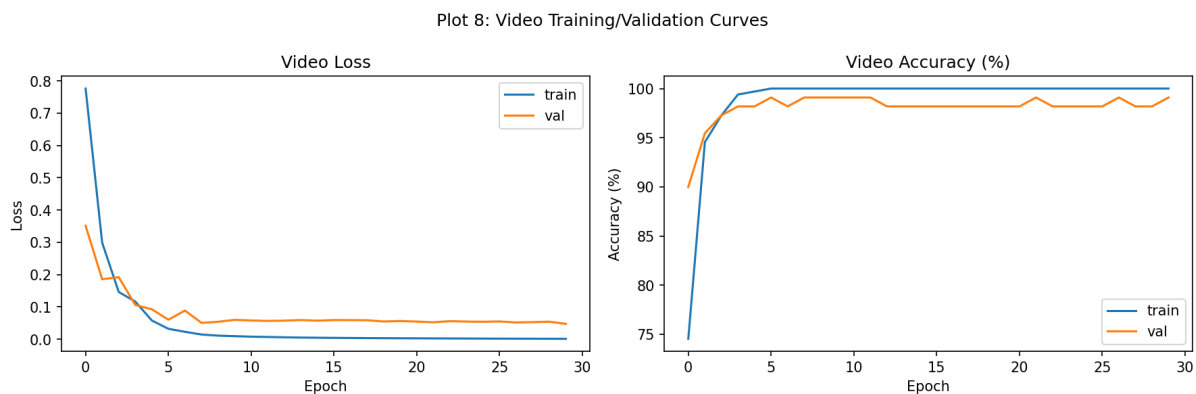


Figure 7: Plot 8: Video training and validation curves.

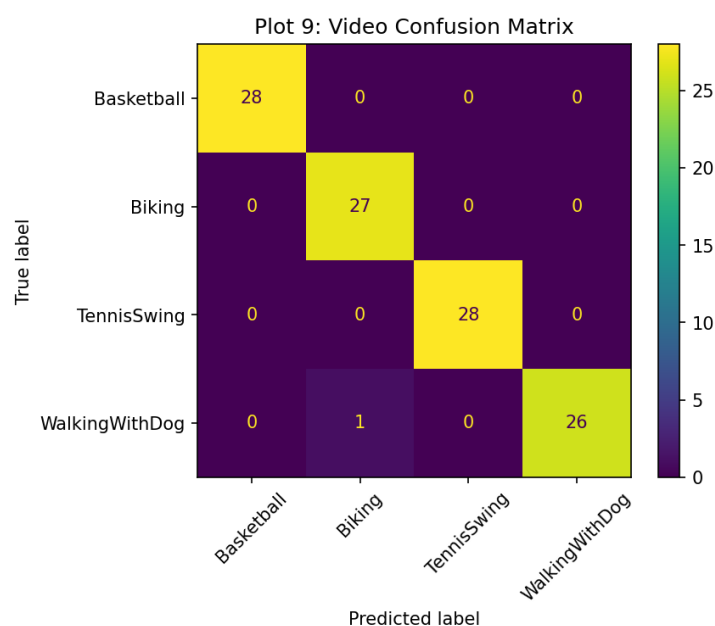


Figure 8: Plot 9: Video confusion matrix.

For a sample test video the model predicted Biking with an actual label of Biking and a confidence of 0.98.

Inference. This CNN captures the spatial content of each frame, such as the players, the ball, the bicycle or the dog and the surrounding scene. The recurrent model which then captures how the content changes across the ten frames, Arranging the CNN features as a sequence is what lets the LSTM or GRU model motion instead of just single frame appearance. The pipeline separates the spatial and temporal parts of the problem cleanly.

12 Sequence to Sequence Learning

A synthetic reversal task was built where an integer sequence maps to its reverse, for example $[3, 8, 1, 5] \rightarrow [5, 1, 8, 3]$. An encoder LSTM compressed the input into a context state and a decoder LSTM generated the reversed output. Several thousand short sequences were used.

Table 7: Sample sequence to sequence predictions.

Input	Predicted	Target
[5, 9, 9, 8, 2, 3]	[3, 2, 8, 9, 9, 5]	[3, 2, 8, 9, 9, 5]
[2, 2, 0, 9, 5, 8]	[8, 5, 9, 0, 2, 2]	[8, 5, 9, 0, 2, 2]
[4, 9, 1, 9, 3, 5]	[5, 3, 9, 1, 9, 4]	[5, 3, 9, 1, 9, 4]
[6, 6, 3, 9, 9, 4]	[4, 9, 9, 3, 6, 6]	[4, 9, 9, 3, 6, 6]
[3, 9, 5, 6, 1, 9]	[9, 1, 6, 5, 9, 3]	[9, 1, 6, 5, 9, 3]

The token accuracy was 99.30 percent and the sequence accuracy was 96.50 percent, with a final validation loss of 0.0249.

Inference. The sequence accuracy is much more stricter as just one token being wrong make the whole sequence wrong .So therefore the token accuracy is always higher or is equal to it .

13 Consolidated Results

Table 8: Consolidated results across all models.

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	76.67	76.64	76.67	76.59	1974
LSTM	94.17	94.32	94.17	94.15	6006
GRU	95.00	95.01	95.00	94.98	4758
CNN-LSTM	99.09	99.11	99.07	99.07	168196

Table 9: Sequence to sequence results.

Measure	Value
Token accuracy (%)	99.30
Sequence accuracy (%)	96.50
Final validation loss	0.0249

The token accuracy is almost 100 percent which is good. It probably reached a high percent like this cause the data wasn't that large and the data was given to be easily separable.

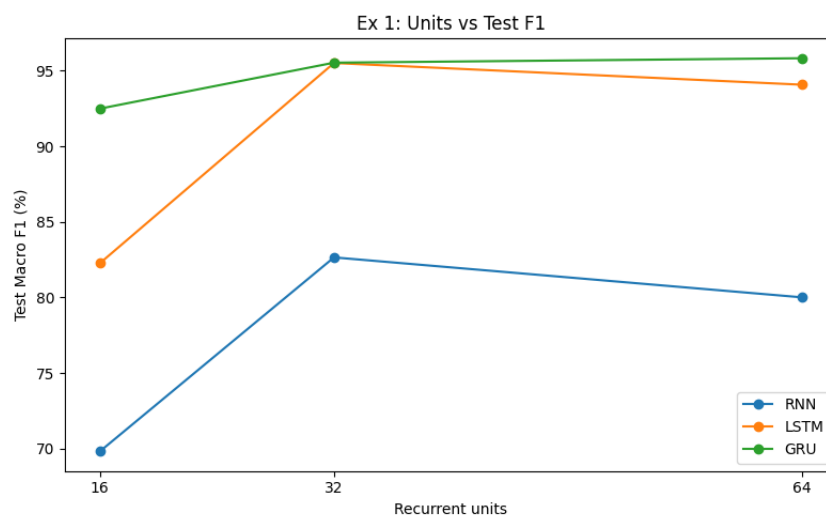
14 Additional Exercises

14.1 Effect of recurrent units

The three models were retrained with 16, 32 and 64 units.

Table 10: Effect of the number of recurrent units.

Model	Units	Accuracy	F1	Parameters	Time (s)
RNN	16	70.00	69.82	790	68.7
LSTM	16	82.22	82.27	2038	10.4
GRU	16	92.50	92.50	1670	9.6
RNN	32	82.78	82.65	1974	68.7
LSTM	32	95.56	95.51	6006	9.6
GRU	32	95.56	95.54	4758	9.3
RNN	64	80.00	80.01	5878	69.2
LSTM	64	94.17	94.08	20086	9.6
GRU	64	95.83	95.83	15542	9.6

**Figure 9:** Additional Exercise 1: Number of units versus test F1.

Inference. With more units the parameters increase a lot along with it ,after 32 units no significant increase in accuracy occurs ,RNN even goes down a little when going from 32 to 64 ,this is probably because of the vanishing gradient problem. SO using 32 units is the best way to go about it .

14.2 GRU versus LSTM at the same units

At 32 units the two models were compared directly. The GRU reached 94.44 accuracy and 94.38 F1 with 4758 parameters, and the LSTM reached 94.44 accuracy and 94.42 F1 with 6006 parameters. The two are really close and the GRU uses fewer parameters even though it gives the same result.

14.3 Second recurrent layer

A stacked two layer version was compared with the single layer models.

Table 11: Single layer versus two layer test F1.

Model	1 layer F1	2 layer F1	2 layer Parameters
RNN	76.59	82.03	4054
LSTM	94.15	96.11	14326
GRU	94.98	96.09	11094

Inference. The second layer improved the F1 accuracy but also increased the parameters so a higher computational cost .

14.4 Bidirectional versus unidirectional LSTM

The bidirectional LSTM reached 94.17 accuracy and 94.12 F1 with 11894 parameters and a training time of 25.2 seconds, compared with 94.17 accuracy and 94.15 F1 for the unidirectional LSTM with 6006 parameters and 14.3 seconds.

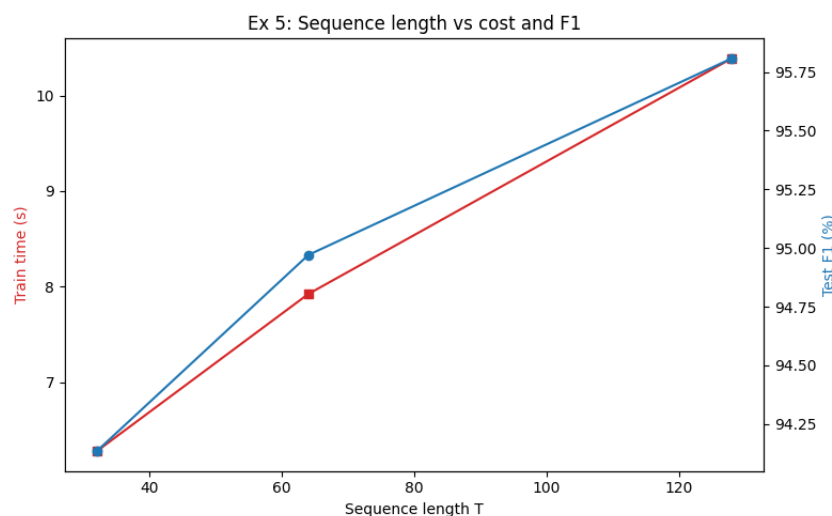
Inference. No improvements were found when implementing the bidirectional model .It only the increased the parameter count .Which makes sense as the full window was always available to us the entire time.So it did not add any useful information here.

14.5 Sequence length versus computational cost

An LSTM was timed at each sequence length.

Table 12: Sequence length versus F1 and training time.

T	F1	Training time (s)
32	94.13	6.3
64	94.97	7.9
128	95.81	10.4

**Figure 10:** Additional Exercise 5: Sequence length versus cost and F1.

Inference. The higher the sequence length the higher the training time ,from the graph we can see it scales linearly .It also increases the F1 accuracy.So basically longer sequence: higher training time , higher F1 score.

14.6 Video LSTM versus GRU on identical features

Using the same frozen CNN features, the CNN-LSTM reached 99.09 percent accuracy and 99.07 percent F1 with 168196 parameters, and the CNN-GRU reached 97.27 percent accuracy and 97.26 percent F1 with 126276 parameters. Both trained in about 6 seconds.

Inference. Since both models received the same features, any difference comes only from the recurrent layer. The GRU used fewer parameters and came very close, with the LSTM slightly ahead on this split, which matches the pattern seen on the sensor data.

14.7 Sequence to sequence with a different output length

The task was changed so the output was one token longer than the input, by appending a copy of the last reversed token, giving a length 6 to length 7 mapping. The model got 99.31 token accuracy and 95.50 sequence accuracy

Inference.

The decoder length is set independently of the input length, so the model can map a length of 6 input to a length 7 output. This shows the general encoder decoder property that the context state fixes what is remembered and the decoder can emit a sequence of any chosen length.

15 Discussion Questions

1. A recurrent neural network processes a sequence by keeping a hidden state that is updated at each step, with the same weights reused across time.
2. The hidden state holds a summary of everything the network has seen up to the current step.
3. Sequence order matters because the meaning of the data depends on how values change over time, and reordering destroys that structure.
4. A feed forward network treats each input on its own, while an RNN feeds its previous state back in, which gives it memory.
5. Backpropagation Through Time is backpropagation applied to the unrolled network, sending the error backward through the time steps.
6. Gradients vanish because many derivative terms below one are multiplied together over long sequences.

7. Gradients explode when the recurrent weights are large, so the repeated products grow without bound.
8. Long term dependencies are relationships between time steps that are far apart in the sequence.
9. The LSTM cell state is a protected memory channel that carries information across many steps with little decay.
10. The forget gate discards old memory, the input gate writes new information and the output gate decides what part of the memory becomes the hidden state.
11. The reset gate controls how much past state to ignore and the update gate blends the old state with the new candidate state.
12. LSTM and GRU both handle long dependencies, but the GRU merges the cell and hidden state and uses fewer gates.
13. The GRU is simpler because it combines the forget and input roles into one update gate and drops the separate cell state.
14. The three models must use the same settings so the architecture is the only variable in the comparison.
15. The confusion matrix shows which classes are confused with each other, which plain accuracy does not.
16. Macro F1 averages the per class F1 equally, so small classes are not hidden by large ones.
17. The sequence length sets how much temporal context the model can use.
18. A CNN can be used as a feature extractor because it captures the spatial content of each frame.
19. The CNN extracts spatial information such as shapes and object position.
20. The recurrent network learns the temporal information, meaning how the frame content changes over time.
21. CNN features are arranged as a sequence so the recurrent model can learn the order and motion across frames.
22. In sequence to sequence learning the encoder compresses the input into a context state and the decoder generates the output step by step.
23. Teacher forcing feeds the true previous token, rather than the model prediction, as the decoder input during training.
24. Token accuracy is the fraction of correct tokens and sequence accuracy is the fraction of fully correct sequences.

25. The test set must stay untouched during model selection so it gives an unbiased estimate of how the model generalizes.

16 Conclusion

The experiment shows the full sequence learning pipeline from raw data (gathered) and running it thorough : preprocessing, RNN, LSTM and GRU models and evaluation. The GRU and LSTM did way better than the the Vanilla RNN on the activity recognition task, with the GRU reaching the best test F1 at a lower parameter count than the LSTM. The confusion matrices generated showed that the static activities are the hardest to separate, and the sequence length study showed that the gated models make better use of longer context. The CNN and recurrent pipeline showed the spatial and temporal can be combined into one and the sequence to sequence task showed the encoder decoder architecture ,also the difference between token and sequence accuracy.